

Problem Set 01 — Starting Up

Corresponds to: 01-intro — Module orientation, reading order, bibliography.

Fall in Cleveland. You have two compilers on your machine and a working copy in `practice/tcc/`. This problem set gets you building, running, and looking — not yet understanding. The goal is orientation, not mastery.

Allow yourself three hours, unhurried.

Problem 1 — Build TCC [hands-on, 15 min]

Navigate to `practice/tcc/`. Build it:

```
./configure && make
```

Verify the build succeeded:

```
./tcc --version
```

Record the version number and the date of the build.

If the build fails, read the error output carefully. The Makefile is readable — open it and find the `all:` target before you search the internet. Most build failures on Linux are a missing header or library. Report what went wrong before you fix it.

Write down: version number, build time, any errors encountered and how you resolved them.

Problem 2 — Hello World [hands-on, 10 min]

Navigate to `practice/scratch/`. A starter `hello.c` is already there. Compile it two ways:

```
# With TCC
../tcc/tcc hello.c -o hello-tcc
./hello-tcc

# With the system GCC
gcc hello.c -o hello-gcc
./hello-gcc
```

Both should print the same thing. Now compare:

```
ls -lh hello-tcc hello-gcc
```

Write down: the binary sizes. What do you notice? What might explain the difference?

Problem 3 — Script mode [hands-on, 5 min]

TCC can compile in memory and execute immediately:

```
../tcc/tcc -run hello.c
```

Modify `hello.c` to print your name and the current date. Run it again. Notice the turnaround time between editing and running.

Write down: how does this compare to the `gcc` → binary → run cycle? When would this matter?

Problem 4 — Read the README [reading, 10 min]

Open `02/tcc/README` (the original, not the practice copy). Read it end to end.

Write down:

- What platforms does TCC claim to support?
 - What does TCC claim as its primary advantage over GCC?
 - What is the one-sentence description of TCC that Bellard offers?
-

Problem 5 — File survey [reading, 20 min]

List all `.c` files in `practice/tcc/` with `ls *.c`. For each file in the list below, write a one-sentence guess of its purpose based only on the filename. Do not open the files yet.

<code>tcc.c</code>	<code>tccpp.c</code>	<code>tccgen.c</code>	<code>libtcc.c</code>
<code>tccllf.c</code>	<code>tccrun.c</code>	<code>tcctools.c</code>	<code>tccdbg.c</code>
<code>x86_64-gen.c</code>	<code>i386-gen.c</code>	<code>arm-gen.c</code>	<code>tccasm.c</code>

Now open each file for exactly 30 seconds. Read only enough to revise your guess. Revise.

Write down: your before and after guesses for each file. Which surprises you most?

Note: `tcc.c` is only 432 lines. Does its size match what you expected from the name? Look at lines 22–29 to understand why.

Problem 6 — Primary source [research, 30 min]

Find "The Development of the C Language" by Dennis M. Ritchie (1993). It is freely available online — search the exact title. Read sections 1 through 3.

Write down:

What language came before C? What machine did it run on?

What was the key limitation of that language that motivated C's design?

What does Ritchie say about the relationship between C and Unix?

Write one paragraph per question, in your own words. Cite the section you're drawing from.

Problem 7 — Bibliography check [research, 15 min]

The intro document lists freely available and standard references. For each item below, verify you can actually find and access it:

- Ritchie's development paper (from Problem 6)
- Thompson's "Reflections on Trusting Trust" — *CACM* 27.8 (1984)
- Unix Heritage Society at tuhs.org — can you find the V6 C compiler source?
- Computer History Museum oral histories — find a video interview with Brian Kernighan

Write down: direct links or locations for each. Note anything you couldn't find.

Problem 8 — Reflection [writing, 15 min]

In your own words: TCC and GCC are both C compilers. What is the fundamental difference in their philosophy? Give one concrete example from the code or documentation of each that illustrates this difference.

One paragraph. No more than 200 words.

Next: *Problem Set 02 — the compiler lineage*. Open `ps-02-ontogeny.pdf` and `practice/tcc/tccgen.c`.
